



Министерство науки и высшего образования Российской Федерации
Федеральное автономное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 11
по курсу «Методы оптимизации»
«Метод Дэвидона-Флэтчера-Пауэла (Д.Ф.П.)»

Студент группы ИУ9-81Б Кириленко У.О.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовать метод Дэвидона-Флэтчера-Пауэла (Д.Ф.П.) для функций Розенброка, Растригина, Швевеля.

2 Результаты

Исходный код программы представлен в листингах 1– 1.

Листинг 1: Основное задание

```
1 using LinearAlgebra
2 using Plots
3
4 plotly()
5
6 function rosenbrock(x)
7     return (1.0 - x[1])^2 + 100.0 * (x[2] - x[1]^2)^2
8 end
9
10 function rastrigin(x)
11     return 20.0 + x[1]^2 - 10.0*cos(2*pi*x[1]) + x[2]^2 - 10.0*cos(2*pi*
12     x[2])
13 end
14 function schwefel(x)
15     return 418.9829*2 - (x[1]*sin(sqrt(abs(x[1]))) + x[2]*sin(sqrt(abs(x
16     [2]))))
17 end
18 function get_gradient(f, x; h=1e-6)
19     n = length(x)
20     grad = zeros(n)
21     for i in 1:n
22         x_plus = copy(x); x_plus[i] += h
23         x_minus = copy(x); x_minus[i] -= h
24         grad[i] = (f(x_plus) - f(x_minus)) / (2*h)
25     end
26     return grad
27 end
28
29 #
30 function newton_1d(g, alpha0=0.0; tol=1e-5, max_iter=30, h=1e-5)
31     alpha = alpha0
32     for iter in 1:max_iter
```

```

33         df = (g(alpha + h) - g(alpha - h)) / (2 * h)
34         d2f = (g(alpha + h) - 2 * g(alpha) + g(alpha - h)) / (h^2)
35         if abs(d2f) < 1e-8 break end
36         step = df / d2f
37         alpha = alpha - step
38         if abs(step) < tol || abs(df) < tol break end
39     end
40     return alpha
41 end
42
43 # DFP
44 function dfp(f, x0; max_iters=100, epsilon=1e-4)
45     x = copy(x0)
46     trajectory = [copy(x)]
47     n = length(x)
48
49     H = Matrix{Float64}(I, n, n)
50     grad = get_gradient(f, x)
51
52     for i in 1:max_iters
53         if norm(grad) < epsilon break end
54
55         #
56         p = -H * grad
57
58         #
59         g(alpha) = f(x + alpha * p)
60         alpha = newton_1d(g, 0.0)
61
62         x_new = x + alpha * p
63         grad_new = get_gradient(f, x_new)
64
65         #
66         delta_x = x_new - x
67         delta_g = grad_new - grad
68
69         #
70         dot_dx_dg = dot(delta_x, delta_g)
71         if abs(dot_dx_dg) > 1e-10 && dot_dx_dg > 0
72             A = (delta_x * delta_x') / dot_dx_dg
73
74             z = H * delta_g
75             dot_z_dg = dot(delta_g, z)
76
77             if abs(dot_z_dg) > 1e-10
78                 B = (H * delta_g) * (delta_g' * H) / dot_z_dg

```

```

79
80         #                                     H
81         H = H + A - B
82     else
83         H = H + A
84     end
85 end
86
87 x = x_new
88 grad = grad_new
89 push!(trajectory, copy(x))
90
91 if minimum(eigvals(H)) <= 0
92     @warn "                                     H
93
94                                     $i"
95     H = Matrix{Float64}(I, n, n)
96 end
97
98 end
99
100 function visualize_all_methods(f, name, x0, bounds_pad=2.0)
101     println("                                     : $name |                                     : $x0")
102
103     # DFP
104     t1 = time()
105     x_dfp, hist_dfp = dfp(f, x0)
106     time_dfp = round(time() - t1, digits=6)
107     iters1 = length(hist_dfp) - 1
108
109     println("\nDFP (                                     -                                     - ):")
110     println("                                     = $iters1 |                                     = $(round.(
x_dfp, digits=6)) | f(x) = $(round(f(x_dfp), digits=10)) |
                                     = $time_dfp                                     ")
111
112     #
113
114     all_points = vcat(hist_dfp)
115     all_x = [p[1] for p in all_points]
116     all_y = [p[2] for p in all_points]
117
118     min_x = minimum(all_x) - bounds_pad
119     max_x = maximum(all_x) + bounds_pad
120     min_y = minimum(all_y) - bounds_pad

```

```

120     max_y = maximum(all_y) + bounds_pad
121
122     #
123     xr = range(min_x, max_x, length=100)
124     yr = range(min_y, max_y, length=100)
125     Z = [f([x, y]) for y in yr, x in xr]
126
127     # 2D
128     p1 = contour(xr, yr, Z, levels=50, color=:grays, fill=false,
129                 title="$name (2D", xlabel="x1",
130                 ylabel="x2")
131
132     # DFP
133     plot!(p1, [p[1] for p in hist_dfp], [p[2] for p in hist_dfp],
134            color=:blue, lw=2, marker=:circle, markersize=3, label="DFP")
135     scatter!(p1, [x_dfp[1]], [x_dfp[2]], color=:red, markersize=2,
136             marker=:diamond, label="DFP")
137
138     # 3D
139     p2 = surface(xr, yr, Z, color=:grays, alpha=0.5, colorbar=false,
140                 title="3D", xlabel="x1", ylabel
141                 ="x2", zlabel="f(x)")
142
143     # 3D
144     z_vals_dfp = [f(p) for p in hist_dfp]
145
146     plot!(p2, [p[1] for p in hist_dfp], [p[2] for p in hist_dfp],
147           z_vals_dfp,
148           color=:blue, lw=3, label="DFP")
149
150     scatter!(p2, [x0[1]], [x0[2]], [f(x0)], color=:red, markersize=2,
151             marker=:diamond, label="")
152
153     display(plot(p1, p2, layout=(1,2), size=(1400, 600)))
154
155     return x_dfp
156 end
157
158 visualize_all_methods(rosenbrock, "rosenbrock", [-1.2, 1.0],
159                       0.5)
160 visualize_all_methods(rastrigin, "rastrigin", [1.2, 1.5], 1.0)
161 visualize_all_methods(schwefel, "schwefel", [400.0, 400.0], 100.0)

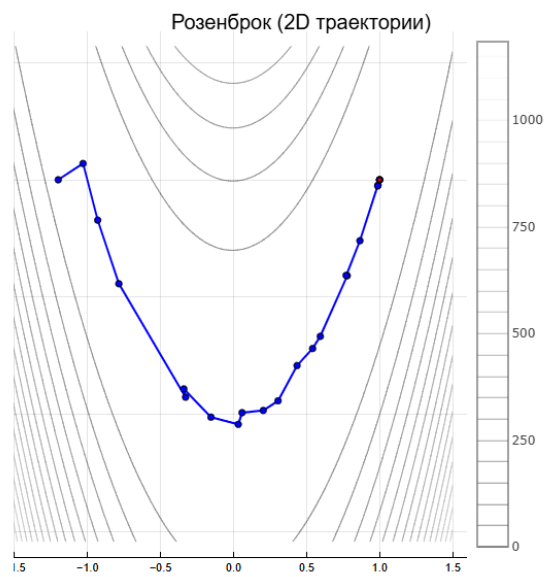
```

Результат запуска представлен на рисунке 1– 3.

Функция: Розенброк | Старт: [-1.2, 1.0]

DFP (Дэвидон-Флетчер-Пауэлл):

Итераций = 21 | Минимум = [0.999998, 0.999997] | $f(x) = 0.0$ | Время = 0.004 сек



3D Поверхность

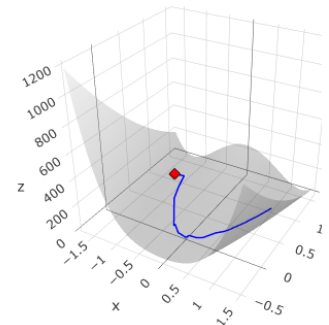
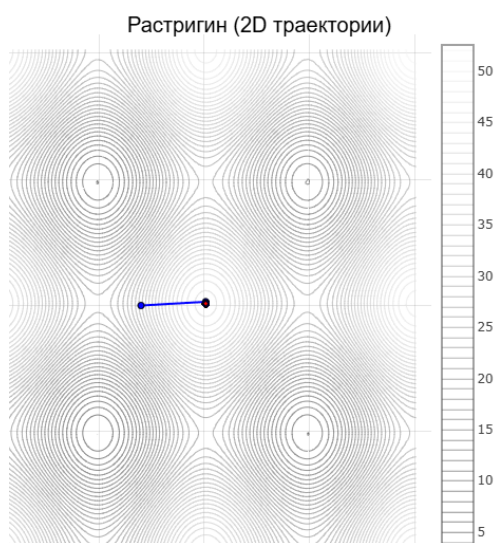


Рис. 1 — Розенброк

Функция: Растрингин | Старт: [1.2, 1.5]

DFP (Дэвидон-Флетчер-Пауэлл):

Итераций = 100 | Минимум = [1.507641, 1.507641] | $f(x) = 44.5229177696$ | Время = 0.002 сек



3D Поверхность

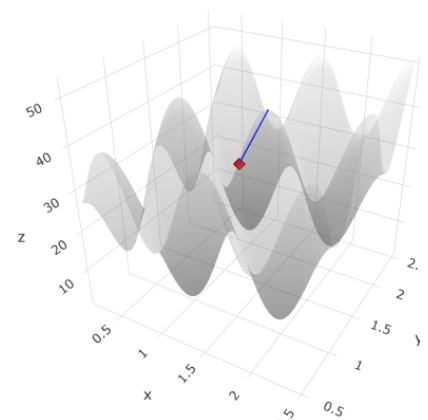


Рис. 2 — Растрингин

Функция: Швевель | Старт: [400.0, 400.0]

DFP (Дэвидон-Флетчер-Пауэлл):

Итераций = 1 | Минимум = [420.968746, 420.968746] | $f(x) = 2.54551e-5$ | Время = 0.0 сек

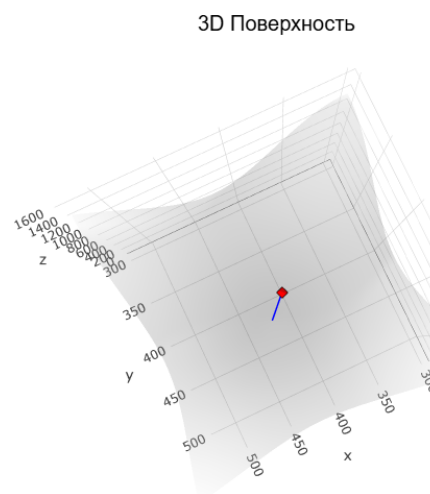
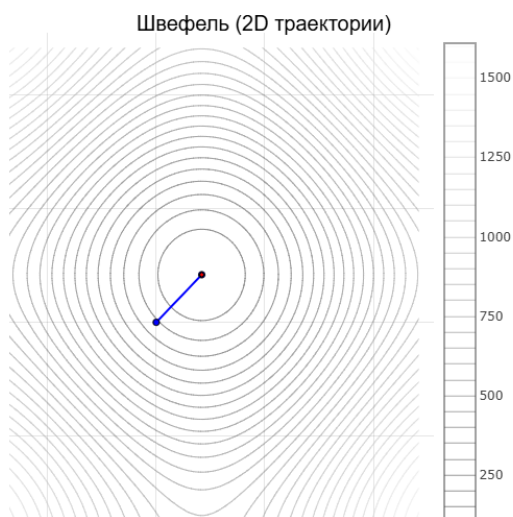


Рис. 3 — Швевель